CHAPTER 04. 데이터프레임으로 2차원 배열 다루기

목차

1. iris 데이터셋
2. 데이터프레임 다루기
3. csv 파일 조회

01 iris 데이터셋

개요

- 데이터프레임(DataFrame)은 2차원 배열 데이터를 저장하고 처리하기 위해 판다스에서 제공하는 자료구조이다.
- 1차원 자료구조인 시리즈(Series)를 여러 개 묶어 놓은 형태이다.
- 행의 인덱스는 index 속성으로, 열의 인덱스는 columns 속성으로 관리된다.

iris 데이터셋

- 데이터프레임 학습을 위한 대표적인 학습용 데이터셋
- 150개의 붓꽃에 대해 4개 분야의 측정 데이터와 품종 정보를 결합한 데이터셋
- 붓꽃 품종: setosa, versicolor, virginica (총 3가지)

01 iris 데이터셋

표 4-1 iris 데이터셋의 컬럼들

컬럼명	의미	자료형(data type)
Sepal_Length	꽃받침의 길이	숫자형
Sepal_Width	꽃받침의 폭	숫자형
Petal_Length	꽃잎의 길이	숫자형
Petal_Width	꽃잎의 폭	숫자형
Species	붓꽃의 품종	문자형(범주형)

	A	B	C	D	E
1	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
2	5.1	3.5	1.4	0.2	setosa
3	4.9	3	1.4	0.2	setosa
4	4.7	3.2	1.3	0.2	setosa
5	4.6	3.1	1.5	0.2	setosa
6	5	3.6	1.4	0.2	setosa
7	5.4	3.9	1.7	0.4	setosa

그림 4-1 iris 데이터셋의 일부

01 iris 데이터셋

▪ 데이터셋 파일 읽기

```
[코드 4-1]
import pandas as pd


df = pd.read_csv('d:/data/iris.csv')          # csv 파일 읽기
df                                            # 내용 확인
```

- .csv 파일은 판다스의 클래스 메소드 pd.read_csv()를 이용하여 읽는다.
- 컬럼 정보가 없는 .csv 파일을 읽을 때는 pd.read_csv('파일명', header=None)와 같이 header=None를 설정한다.
  - 그렇지 않으면 첫 줄에 있는 데이터 값을 컬럼 이름으로 인식

01 iris 데이터셋

```
>>> df                                                    # 내용 확인
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            ...          ...           ...          ...        ...
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

- 150행 50열의 데이터가 df에 저장된다.
- 데이터프레임을 .csv 파일로 저장하려면 to_csv() 메서드를 사용한다.
  - df.to_csv('d:/data/iris_new.csv')

■ 데이터프레임 객체의 정보 확인

```
[코드 4-2]
# 코드4-1에 이어서 실행
# 데이터프레임 객체의 정보 확인
df.info()                                   # 배열의 정보
df.shape                                    # 배열의 형태
df.shape[0]                                 # 행의 개수
df.shape[1]                                 # 컬럼의 개수

df.dtypes                                   # 각 컬럼의 자료형
df['Species'] = df['Species'].astype('category')
df.dtypes

df.columns                                  # 컬럼들의 이름 보기
df.head()                                   # 데이터 앞부분 일부 보기
df.tail()                                   # 데이터 뒷부분 일부 보기
df['Species'].unique()                      # 품종 정보 확인
```

02 데이터프레임 다루기

```
>>> df.info()                                    # 배열의 정보
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Sepal_Length  150 non-null    float64
 1   Sepal_Width   150 non-null    float64
 2   Petal_Length  150 non-null    float64
 3   Petal_Width   150 non-null    float64
 4   Species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

- info() 메서드는 데이터프레임 객체에 대한 전반적인 정보를 알려준다.

02 데이터프레임 다루기

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Sepal_Length  150 non-null    float64
 1   Sepal_Width   150 non-null    float64
 2   Petal_Length  150 non-null    float64
 3   Petal_Width   150 non-null    float64
 4   Species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

df의 자료형은 데이터프레임

df에는 150개의 행이 있음

df에는 5개의 컬럼이 있음

컬럼에 대한 정보

컬럼의 이름

결측값이 아닌 값의 개수

컬럼의 자료형

그림 4-2 데이터프레임 객체의 출력 결과에 대한 설명

02 데이터프레임 다루기

```
>>> df.shape                                    # 배열의 형태
(150, 5)
>>> df.shape[0]                                 # 행의 개수
150
>>> df.shape[1]                                 # 컬럼의 개수
5
```

```
>>> df.dtypes                                   # 각 컬럼의 자료형
Sepal_Length    float64
Sepal_Width     float64
Petal_Length    float64
Petal_Width     float64
Species          object
dtype: object
```

- 앞 4개의 컬럼은 숫자형(float)이고, Species 컬럼은 객체형(object)이다.
- 마지막 줄의 dtype: object는 데이터프레임 전체의 자료형을 나타낸다.

02 데이터프레임 다루기

```
>>> df['Species'] = df['Species'].astype('category')
>>> df.dtypes
Sepal_Length     float64
Sepal_Width      float64
Petal_Length     float64
Petal_Width      float64
Species         category
dtype: object
```

- 판다스는 데이터 파일에서 데이터를 읽어올 때, 컬럼의 자료형을 자동으로 판단한다.
- 문자열 값을 포함하는 컬럼은 기본적으로 object 타입으로 지정된다.
- Species 컬럼은 문자열 값을 저장하지만, 임의의 문자열이 아니라 특정한 종류가 정해진 값이므로 범주형(categorical) 자료이다.
- 범주형 자료는 판다스에서 category라는 별도의 자료형으로 지정할 수 있다.

02 데이터프레임 다루기

- **범주형 자료 컬럼을 category 타입으로 지정하면 좋은 이유**
  - **메모리 절약**
    - 문자열 값을 그대로 저장했을 때보다 메모리를 절약할 수 있다.
  - **논리적 순서 지정**
    - 문자열 값은 출력 순서를 오름차순 또는 내림차순 정도로만 조정할 수 있다.
    - 그러나 category 타입은 알파벳 순서와 무관하게 논리적 순서를 설정할 수 있다.
      - » ex) 계절 컬럼값의 논리적 순서를 Spring → Summer → Fall → Winter로 설정 가능
  - **범주형 정보 전달**
    - 데이터프레임을 다른 파이썬 라이브러리에서 사용할 때, category 타입을 지정하면 해당 컬럼이 범주형 자료라는 정보를 전달할 수 있다.
    - 이를 통해 통계 함수나 그래프 함수에서 해당 컬럼을 범주형 자료로 올바르게 처리할 수 있다.

02 데이터프레임 다루기

```
>>> df.columns                                    # 컬럼들의 이름 보기
Index(['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width',
       'Species'],
      dtype='object')
```

- 데이터프레임의 컬럼 이름은 columns 속성에 저장된다.
- columns 속성은 배열 형태로 저장되므로, df.columns[2]와 같은 인덱싱이 가능하다.
  - df.columns[2]는 'Petal_Length'을 의미한다.

02 데이터프레임 다루기

```
>>> df.head()                                  # 데이터 앞부분 일부 보기
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width Species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
>>> df.tail()                                  # 데이터 뒷부분 일부 보기
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica
```

- 앞부분 10개 행의 데이터를 확인하려면 df.head(10)을 사용한다.
- 뒷부분 20개 행의 데이터를 확인하려면 df.tail(20)을 사용한다.

02 데이터프레임 다루기

```
>>> df['Species'].unique()                    # 품종 정보 확인
['setosa', 'versicolor', 'virginica']
Categories (3, object): ['setosa', 'versicolor', 'virginica']
```

- unique() 메서드는 자료에서 중복된 값을 제거하고, 데이터에 포함된 고유한 값들을 확인하는 데 사용된다.

02 데이터프레임 다루기

■ 데이터프레임 안에서 값의 위치 다루기

▪ 인덱싱(indexing)과 슬라이싱(slicing)

[코드 4-3]

```
# 코드4-1에 이어서 실행
# 인덱싱과 슬라이싱
df.iloc[2,3]                                # 2행 3열의 값
df.loc[3,'Petal_Width']                     # 'Petal_Width' 컬럼의 3행 값
df.loc[[0,2,4],['Petal_Length', 'Petal_Width']]

df.loc[5:8,'Petal_Length']                  # 'Petal_Length' 컬럼의 5~7행 값
df.iloc[:5,:4]                              # 0~4행, 0~3열의 값

df.loc[:, 'Petal_Length']                   # 'Petal_Length' 컬럼의 모든 행의 값
df['Petal_Length']                          # 'Petal_Length' 컬럼의 모든 값
df.Petal_Length                             # 'Petal_Length' 컬럼의 모든 값

df.iloc[:5,:]                               # 0~4행의 모든 컬럼의 값
df.iloc[:5]                                 # 0~4행의 모든 컬럼의 값
```

02 데이터프레임 다루기

```
>>> df.iloc[2,3]                                # 2행 3열의 값
np.float64(0.2)
>>> df.loc[3,'Petal_Width']                     # 'Petal_Width' 컬럼의 3행 값
np.float64(0.2)
```

- 절대 위치 인덱스를 사용할 때는 iloc[], 레이블 인덱스를 사용할 때는 loc[]를 사용한다.
- 데이터프레임은 2차원 배열이므로, iloc[2,3]처럼 행과 열을 지정하는 두 개의 인덱스가 필요하다.
  - 앞쪽은 행 인덱스, 뒤쪽은 열(컬럼) 인덱스를 의미한다.

```
>>> df.loc[[0,2,4],['Petal_Length', 'Petal_Width']]
   Petal_Length  Petal_Width
0           1.4          0.2
2           1.3          0.2
4           1.4          0.2
```

- 여러 개의 행과 컬럼을 지정할 때는 리스트([ ]로 묶어서 지정한다.

02 데이터프레임 다루기

```
>>> df.loc[5:8,'Petal_Length']                    # 'Petal_Length' 컬럼의 5~7행 값
5    1.7
6    1.4
7    1.5
8    1.4
Name: Petal_Length, dtype: float64
>>> df.iloc[:5,:4]                                # 0~4행, 0~3열의 값
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
0           5.1          3.5           1.4          0.2
1           4.9          3.0           1.4          0.2
2           4.7          3.2           1.3          0.2
3           4.6          3.1           1.5          0.2
4           5.0          3.6           1.4          0.2
```

5~7이아닌 5~8 임에 주의

- 데이터프레임의 인덱싱에서도 범위 지정 연산자(:)를 사용할 수 있다.
  - df.loc[5:8, 'Petal_Length']에서 사용된 인덱스는 절대 위치 인덱스가 아니라 레이블 인덱스이다.

02 데이터프레임 다루기

```
>>> df.loc[:, 'Petal_Length']                    # 'Petal_Length' 컬럼의 모든 행의 값
0      1.4
1      1.4
2      1.3
3      1.5
...
146    5.0
147    5.2
148    5.4
149    5.1
Name: Petal_Length, Length: 150, dtype: float64
>>> df['Petal_Length']                          # 'Petal_Length' 컬럼의 모든 값
0      1.4
1      1.4
2      1.3
3      1.5
...
146    5.0
147    5.2
148    5.4
149    5.1
```

02 데이터프레임 다루기

```
>>> df.Petal_Length                          # 'Petal_Length' 컬럼의 모든 값
0      1.4
1      1.4
2      1.3
3      1.5
4      1.4
      ...
145    5.2
146    5.0
147    5.2
148    5.4
149    5.1
Name: Petal_Length, Length: 150, dtype: float64
```

- 특정 컬럼의 모든 행 값을 추출할 때는 df.loc[:, 'Petal_Length']를 사용할 수 있다.
- 보다 간단하게 df['Petal_Length'] 형태로도 접근 가능하다.
- 또한, df.Petal_Length 형태로도 접근할 수 있다. (단, 컬럼 이름에 공백이나 특수문자가 포함된 경우에는 사용할 수 없음)

02 데이터프레임 다루기

```
>>> df.iloc[:5,:]                                # 0~4행의 모든 컬럼의 값
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width Species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
>>> df.iloc[:5]                                  # 0~4행의 모든 컬럼의 값
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width Species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

- 특정 행의 모든 컬럼 값을 추출하는 방법은 두 가지가 있다.

– df.iloc[:5, :] → 컬럼 인덱스 부분에 전체 범위를 의미하는 : 사용

– df.iloc[:5] → 컬럼 인덱스 부분을 생략

» iloc[]에서 인덱스를 하나만 지정하면, 이는 행의 인덱스를 의미한다.

[여기서 잠깐!] 데이터 분석에 들어가는 시간

- 특정 컬럼의 모든 행의 값들을 추출

```
df.loc[:, 'Petal_Length']                    # 정식 표현
df['Petal_Length']                           # 약식 표현
df.Petal_Length                              # 약식 표현
```

- 특정 행의 모든 컬럼의 값들을 추출

```
df.iloc[:5,:]                                # 정식 표현
df.iloc[:5]                                  # 약식 표현
```

02 데이터프레임 다루기

▪ 조건문을 이용한 슬라이싱

- 데이터프레임에서 조건문은 주로 조건에 맞는 행을 추출하는 데 사용된다.
- 따라서, 조건문은 일반적으로 행의 인덱스 입력 부분에 작성한다.

[코드 4-4]

```
# 코드4-1에 이어서 실행

# 꽃잎의 길이가 6.5 이상인 행들의 모든 컬럼을 보이시오
df.loc[df.Petal_Length >= 6.5, :]
df.loc[df.Petal_Length >= 6.5]          # 모든 컬럼의 경우 컬럼 인덱스 생략

# 꽃잎의 길이가 6.5 이상인 행들의 인덱스 번호를 보이시오
df.loc[df.Petal_Length >= 6.5].index
```

02 데이터프레임 다루기

- [코드 4-4](계속)

```
# 꽃잎의 길이가 3.5~3.8 사이인 행들의 모든 컬럼을 보이시오
df.loc[(df.Petal_Length >= 3.5) &
       (df.Petal_Length <= 3.8)]

# 꽃잎의 길이가 1.3 미만이거나 6.5 를 초과하는 행들의 꽃잎의 길이와 폭을 보이시오
df.loc[(df.Petal_Length < 1.3) |
       (df.Petal_Length > 6.5), ['Petal_Length','Petal_Width']]

# where() 를 이용한 조건 검색
df.where(df.Petal_Length >= 6.5).dropna()
```

02 데이터프레임 다루기

```
>>> # 꽃잎의 길이가 6.5 이상인 행들의 모든 컬럼을 보이시오
>>>
>>> df.loc[df.Petal_Length >= 6.5, :]
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
105           7.6          3.0           6.6          2.1  virginica
117           7.7          3.8           6.7          2.2  virginica
118           7.7          2.6           6.9          2.3  virginica
122           7.7          2.8           6.7          2.0  virginica
```

- 조건문은 행의 인덱스 위치에 작성한다.
- 열 인덱스에 :를 사용하면 모든 컬럼을 의미한다.
- 결과에서 왼쪽에 표시된 105, 117, 118, 122는 추출된 행들의 원본 데이터셋에서의 인덱스 번호이다.

02 데이터프레임 다루기

```
>>> df.loc[df.Petal_Length >= 6.5]        # 모든 컬럼의 경우 컬럼 인덱스 생략
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
105           7.6          3.0           6.6          2.1  virginica
117           7.7          3.8           6.7          2.2  virginica
118           7.7          2.6           6.9          2.3  virginica
122           7.7          2.8           6.7          2.0  virginica
```

```
>> # 꽃잎의 길이가 6.5 이상인 행들의 인덱스 번호를 보이시오
>>>
>>> df.loc[df.Petal_Length >= 6.5].index
Index([105, 117, 118, 122], dtype='int64')
```

02 데이터프레임 다루기

```
>>> # 꽃잎의 길이가 3.5~3.8 사이인 행들의 모든 컬럼을 보이시오
>>>
>>> df.loc[(df.Petal_Length >= 3.5) &
...        (df.Petal_Length <= 3.8)]
    Sepal_Length  Sepal_Width  Petal_Length  Petal_Width     Species
60           5.0          2.0           3.5          1.0  versicolor
64           5.6          2.9           3.6          1.3  versicolor
79           5.7          2.6           3.5          1.0  versicolor
80           5.5          2.4           3.8          1.1  versicolor
81           5.5          2.4           3.7          1.0  versicolor
```

- 다중 조건을 적용할 때는 각 조건을 ( )로 감싸고, &(and) 또는 |(or) 연산자로 연결해야 한다.

02 데이터프레임 다루기

```
>>> # 꽃잎의 길이가 1.5 미만이거나 6.5 를 초과하는 행들의 꽃잎의 길이와 폭을
보이시오
>>>
>>> df.loc[(df.Petal_Length < 1.3) ¦
...        (df.Petal_Length > 6.5), ['Petal_Length','Petal_Width']]
     Petal_Length  Petal_Width
13            1.1          0.1
14            1.2          0.2
22            1.0          0.2
35            1.2          0.2
105           6.6          2.1
117           6.7          2.2
118           6.9          2.3
122           6.7          2.0
```

02 데이터프레임 다루기

```
>>> # where() 를 이용한 조건 검색
>>>
>>> df.where(df.Petal_Length >= 6.5).dropna()
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
105           7.6          3.0           6.6          2.1  virginica
117           7.7          3.8           6.7          2.2  virginica
118           7.7          2.6           6.9          2.3  virginica
122           7.7          2.8           6.7          2.0  virginica
```

- where() 메서드는 각 행이 조건에 부합하는지 판단하고, 부합하지 않는 행의 값을 결측값(NaN)으로 대체한다.
- 이 결과에 dropna() 메서드를 적용하면, 결측값이 없는 행들만 남게 되어 최종적으로 조건에 맞는 행들만 추출된다.

■ 데이터프레임 객체에 대한 연산

▪ 데이터프레임 객체에 대한 산술 연산

- 데이터프레임 객체에 대한 산술 연산은 숫자형 값들에 대해서만 가능하다.
- 데이터프레임에는 숫자형 컬럼과 문자형 컬럼이 혼합되어 있을 수 있으므로, 산술 연산 적용 시 주의해야 한다.
- 기본 원칙: 숫자형 컬럼들에 대해서만 산술 연산이 적용된다.

02 데이터프레임 다루기

[코드 4-5]

```
# 코드4-1에 이어서 실행

# 데이터프레임 객체에 대한 산술 연산
df.loc[:, df.columns != 'Species']                # 'Species' 컬럼을 제외
df.loc[:, ~df.columns.isin(['Species'])]          # 'Species' 컬럼을 제외

df.loc[:, df.columns != 'Species'] + 10           # 모든 값들에 10을 더함

df['Sepal_Length'] + df['Petal_Length']           # 두 컬럼의 같은 행 값들끼리 연산

# 꽃잎의 길이가 5 이상인 경우는 꽃잎의 길이에 -1을 곱하시오
# (5미만인 경우는 원래값 유지)
tmp = df['Petal_Length'].apply(lambda x: -1 if x >= 5 else 1)
df['Petal_Length'] * tmp
```

02 데이터프레임 다루기

```
>>> df.loc[:, df.columns != 'Species']                # 'Species' 컬럼을 제외
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
0             5.1          3.5           1.4          0.2
1             4.9          3.0           1.4          0.2
2             4.7          3.2           1.3          0.2
3             4.6          3.1           1.5          0.2
4             5.0          3.6           1.4          0.2
..            ...          ...           ...          ...
145           6.7          3.0           5.2          2.3
146           6.3          2.5           5.0          1.9
147           6.5          3.0           5.2          2.0
148           6.2          3.4           5.4          2.3
149           5.9          3.0           5.1          1.8

[150 rows x 4 columns]
```

- 조건문 df.columns != 'Species'는 df의 컬럼들 중에서 이름이 'Species'가 아닌 컬럼들을 의미한다.

02 데이터프레임 다루기

```
>>> df.loc[:, ~df.columns.isin(['Species'])]        # 'Species' 컬럼을 제외
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
0             5.1          3.5           1.4          0.2
1             4.9          3.0           1.4          0.2
2             4.7          3.2           1.3          0.2
3             4.6          3.1           1.5          0.2
4             5.0          3.6           1.4          0.2
..            ...          ...           ...          ...
145           6.7          3.0           5.2          2.3
146           6.3          2.5           5.0          1.9
147           6.5          3.0           5.2          2.0
148           6.2          3.4           5.4          2.3
149           5.9          3.0           5.1          1.8

[150 rows x 4 columns]
```

- ~df.columns.isin(['Species'])의 의미는 df의 컬럼들 중 isin()에 지정된 컬럼(즉, 'Species')과 일치하지 않는 컬럼을 선택

02 데이터프레임 다루기

- 다음의 두 명령문은 동일한 결과를 도출
- df.loc[:, (df.columns != 'Sepal_Length’) & (df.columns != 'Petal_Length')]
- df.loc[:, ~df.columns.isin(['Sepal_Length', 'Petal_Length'])]

02 데이터프레임 다루기

```
>>> df.loc[:, df.columns != 'Species'] + 10          # 모든 값들에 10을 더함
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
0            15.1         13.5          11.4         10.2
1            14.9         13.0          11.4         10.2
2            14.7         13.2          11.3         10.2
3            14.6         13.1          11.5         10.2
4            15.0         13.6          11.4         10.2
..            ...          ...           ...          ...
145          16.7         13.0          15.2         12.3
146          16.3         12.5          15.0         11.9
147          16.5         13.0          15.2         12.0
148          16.2         13.4          15.4         12.3
149          15.9         13.0          15.1         11.8

[150 rows x 4 columns]
```

- df 의 내용이 변경되지는 않는다.

02 데이터프레임 다루기

```
>>> df['Sepal_Length'] + df['Petal_Length']    # 두 컬럼의 같은행 값들끼리 연산
0       6.5
1       6.3
2       6.0
3       6.1
...
146    11.3
147    11.7
148    11.6
149    11.0
Length: 150, dtype: float64
```

- 컬럼의 자료형이 숫자형인 경우, 컬럼 간 산술 연산이 가능하다.
- 예제에서는 Sepal_Length(꽃받침 길이) 컬럼과 Petal_Length(꽃잎 길이) 컬럼에 대해 더하기 연산을 수행한다.
- 컬럼 간의 산술 연산은 같은 행 인덱스에 위치한 값들끼리 연산이 수행된다.

02 데이터프레임 다루기

```
>>> # 꽃잎의 길이가 5 이상인 경우는 꽃잎의 길이에 -1을 곱하시오
>>> # (5미만인 경우는 원래값 유지)
>>>
>>> tmp = df['Petal_Length'].apply(lambda x: -1 if x >= 5 else 1)
>>> tmp
0      1
1      1
2      1
3      1
..
146   -1
147   -1
148   -1
149   -1
Name: Petal_Length, Length: 150, dtype: int64
```

02 데이터프레임 다루기

- 람다 함수 lambda x: -1 if x >= 5 else 1의 의미:
  - x 값이 5 이상이면 -1을 반환하고, 그렇지 않으면 1을 반환한다.
- 이 함수가 df['Petal_Length']에 적용되면 Petal_Length 컬럼의 각 값이 x에 입력되며, 조건에 따라 1 또는 -1이 출력된다.

02 데이터프레임 다루기

```
>>> df['Petal_Length'] * tmp
0      1.4
1      1.4
2      1.3
3      1.5
...
146   -5.0
147   -5.2
148   -5.4
149   -5.1
Name: Petal_Length, Length: 150, dtype: float64
```

- Petal_Length 컬럼의 값들과 tmp 변수의 값들을 곱하면, 같은 위치의 값들끼리 연산이 수행된다.
- 그 결과, Petal_Length 값이 5보다 큰 경우에만 -1이 곱해지는 효과를 갖는다.

02 데이터프레임 다루기

▪ 데이터프레임의 통계 관련 메서드

- 데이터프레임의 통계 메서드는 숫자형 타입의 컬럼에 대해서만 적용해야 한다.

[코드 4-6]

```
# 코드4-1에 이어서 실행


df2 = df.loc[:, df.columns != 'Species']     # 숫자타입 컬럼만 추출
df2.sum()                                    # 값들의 합계
df2.mean()                                   # 값들의 평균
df2.median()                                 # 값들의 중앙값
df2.max()                                    # 값들의 최대값
df2.min()                                    # 값들의 최소값
df2.std()                                    # 값들의 표준편차
df2.var()                                    # 값들의 분산
df2.abs()                                    # 값들의 절대값
df2.describe()                               # 기초통계정보
```

02 데이터프레임 다루기

```
>>> df2.sum()                                    # 값들의 합계 (컬럼방향)
Sepal_Length    876.5
Sepal_Width     458.6
Petal_Length    563.7
Petal_Width     179.9
dtype: float64
```

- 데이터프레임에 대한 통계 메서드의 적용은 데이터프레임 안에 있는 4개 컬럼들에 대한 적용으로 변환되어 수행된다.
- 결과에서 첫 번째 행의 Sepal_Length 값 876.5의 의미는 Sepal_Length 컬럼에 있는 값들의 합계가 876.5라는 의미이다.
- df2.sum()의 동일한 표현식은 df2.sum(axis=0)이다.
  - axis는 축의 방향을 의미하며, axis=0은 세로 방향, axis=1은 가로 방향이다.
  - axis는 대부분의 통계 메서드에 적용되며, 이를 이용하여 데이터프레임에서 컬럼별, 행별 통계 값을 계산할 수 있다.

02 데이터프레임 다루기

```
>>> df2.sum(axis=1)                                    # 값들의 합계 (행방향)
0      10.2
1       9.5
2       9.4
3       9.4
4      10.2
       ...
145    17.2
146    15.7
147    16.7
148    17.3
149    15.8
```

02 데이터프레임 다루기

```
>>> df2.mean()                                    # 값들의 평균
Sepal_Length    5.843333
Sepal_Width     3.057333
Petal_Length    3.758000
Petal_Width     1.199333
dtype: float64
>>> df2.median()                                  # 값들의 중앙값
Sepal_Length    5.80
Sepal_Width     3.00
Petal_Length    4.35
Petal_Width     1.30
dtype: float64
>>> df2.max()                                     # 값들의 최대값
Sepal_Length    7.9
Sepal_Width     4.4
Petal_Length    6.9
Petal_Width     2.5
dtype: float64
```

02 데이터프레임 다루기

```
>>> df2.describe()                                    # 기초통계정보
       Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

■ 데이터프레임 객체의 내용 변경

▪ 데이터프레임 객체에 있는 값의 수정

```
[코드 4-7]
# 코드4-1에 이어서 실행

# 데이터프레임 객체에 있는 값의 수정
df3 = df.copy()                                # df를 df3으로 복사
df3.iloc[1,2] = 5.5                            # 1행2열의 값 수정
df3.loc[1,'Petal_Length'] = 1.1

df3.Petal_Length.to_list()                     # 시리즈를 리스트로
df3.loc[df.Petal_Length > 6.5,'Petal_Length'] *= 100
df3.Petal_Length.to_list()
```

02 데이터프레임 다루기

```
>>> # 데이터프레임 객체에 있는 값의 수정
>>>
>>> df3 = df.copy()                                # df를 df3으로 복사
>>> df3.iloc[1,2] = 5.5                            # 1행2열의 값 수정
>>> df3.loc[1,'Petal_Length'] = 1.1
>>> df3.Petal_Length.to_list()                     # 시리즈를 리스트로
[1.4, 1.1, 1.3, 1.5, 1.4, 1.7, 1.4, 1.5, 1.4, 1.5, 1.5, 1.6, 1.4, 1.1, 1.2, 1.5,
(생략)
4.2, 4.3, 3.0, 4.1, 6.0, 5.1, 5.9, 5.6, 5.8, 6.6, 4.5, 6.3, 5.8, 6.1, 5.1, 5.3, 5.5,
5.0, 5.1, 5.3, 5.5, 6.7, 6.9, 5.0, 5.7, 4.9, 6.7, 4.9, 5.7, 6.0, 4.8, 4.9,
(생략)]
>>> df3.loc[df.Petal_Length > 6.5,'Petal_Length'] *= 100
>>> df3.Petal_Length.to_list()
[1.4, 1.1, 1.3, 1.5, 1.4, 1.7, 1.4, 1.5, 1.4, 1.5, 1.5, 1.6, 1.4, 1.1, 1.2, 1.5,
(생략)
4.2, 4.3, 3.0, 4.1, 6.0, 5.1, 5.9, 5.6, 5.8, 660.0, 4.5, 6.3, 5.8, 6.1, 5.1, 5.3,
5.5, 5.0, 5.1, 5.3, 5.5, 670.0, 690.0, 5.0, 5.7, 4.9, 670.0, 4.9, 5.7, 6.0, (생략)]
```

02 데이터프레임 다루기

- **데이터 객체에 대한 행과 컬럼의 추가**

```
[코드 4-8]
# 코드4-7에 이어서 실행
# 데이터프레임 객체에 대한 행과 컬럼의 추가

# 뒷부분에 새로운 행 추가
new_idx = df3.shape[0]                          # 추가할 행번호 결정
df3.loc[new_idx] = [1.1, 4.5, 3.4, 2.2, 'setosa']
df3.tail()

# 중간에 새로운 행 추가
new_row =  pd.DataFrame([[1.1, 2.2, 3.3, 4.4,'virginica']],
                        columns=df3.columns)
new_row
df3 = pd.concat([df3.iloc[:10], new_row, df3.iloc[10:]], ignore_index = True)
df3.iloc[8:13,:]                                # 입력결과 확인
```

02 데이터프레임 다루기

- [코드 4-8](계속)

```
# 여러 행의 추가
ext = pd.DataFrame([[1.2, 3.5, 4.3, 3.1, 'setosa'],
                    [2.1, 3.2, 2.3, 5.2, 'versicolor']],
                   columns=df3.columns)
ext
df3 = df3._append(ext, ignore_index = True)
df3

# 뒤쪽에 컬럼의 추가
new_col = df3.Petal_Length *10
df3['new_col'] = new_col
df3

# 중간에 컬럼의 추가
df4 = df.copy()
df4.insert(loc=2, column='new_col2’, value=new_col)          # 2번째 위치에 추가
Df4
```

02 데이터프레임 다루기

```
>>> # 뒷부분에 새로운 행 추가
>>>
>>> new_idx = df3.shape[0]                          # 추가할 행번호 결정
```

- 추가할 행의 번호는 데이터프레임의 행 개수(df3.shape[0])와 일치해야 한다.

```
>>> df3.loc[new_idx] = [1.1, 4.5, 3.4, 2.2, 'setosa']
>>> df3.tail()
    Sepal_Length Sepal_Width Petal_Length Petal_Width    Species
146          6.3         2.5          5.0         1.9  virginica
147          6.5         3.0          5.2         2.0  virginica
148          6.2         3.4          5.4         2.3  virginica
149          5.9         3.0          5.1         1.8  virginica
150          1.1         4.5          3.4         2.2     setosa
```

- 행을 추가할 때는 df3.iloc[]가 아니라 df3.loc[]을 사용해야 한다.
- new_idx 값 150은 절대 위치 인덱스이면서 동시에 레이블 인덱스이다.

02 데이터프레임 다루기

```
>>> # 중간에 새로운 행 추가 (10번째 위치)
>>>
>>> new_row =  pd.DataFrame([[1.1, 2.2, 3.3, 4.4,'virginica']],
...                          columns=df3.columns)
>>> new_row
  Sepal_Length Sepal_Width Petal_Length Petal_Width    Species
0          1.1         2.2          3.3         4.4  virginica
```

- 중간에 새로운 행을 추가할 때는 단순한 리스트 형태로 추가할 수 없으며, 완전한 데이터프레임 형태로 준비해야 한다.

```
>>> df3 = pd.concat([df3.iloc[:10], new_row, df3.iloc[10:]], ignore_index = True)
```

- 데이터프레임을 중간에 직접 삽입하는 전용 메서드는 존재하지 않는다.
- 따라서, pd.concat() 메서드를 활용해야 한다.
- pd.concat()는 형태가 같은 여러 개의 데이터프레임을 하나로 묶어 새로운 데이터프레임을 생성하는 메서드이다.

02 데이터프레임 다루기

- pd.concat([A, B, C])의 의미는 A, B, C 세 개의 데이터프레임을 차례로 묶어 D라는 하나의 데이터프레임으로 만드는 것이다.

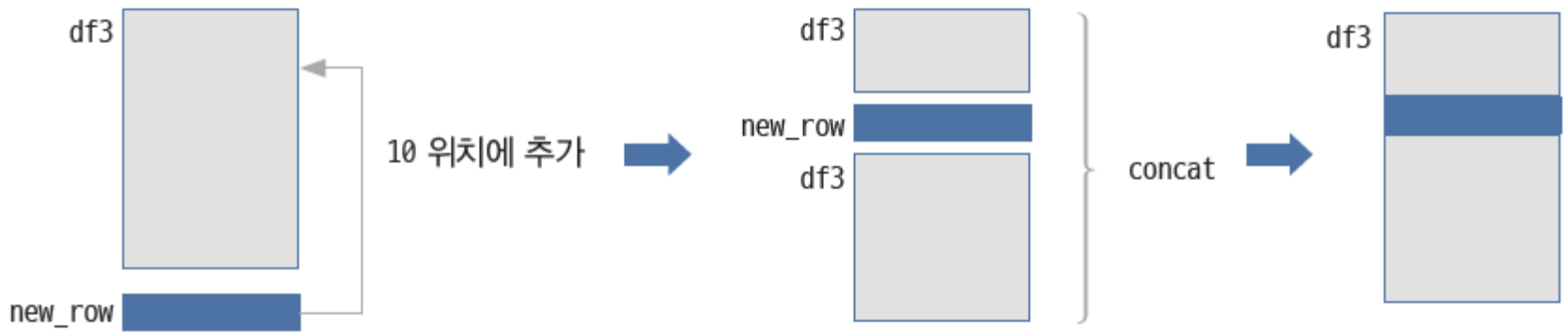


그림 4-3 새로운 행을 중간에 추가하는 과정

```
>>> df3 = pd.concat([df3.iloc[:10], new_row, df3.iloc[10:]], ignore_index = True)
```

- ignore_index = True
  - 여러 개의 데이터프레임을 하나로 묶을 때, 기존 데이터프레임의 인덱스를 무시하고 새로운 일련번호 인덱스를 부여하라는 의미이다.

02 데이터프레임 다루기

```
>>> df3.iloc[8:13,:]                                    # 입력결과 확인
 Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
8           4.4         2.9           1.4          0.2     setosa
9           4.9         3.1           1.5          0.1     setosa
10          1.1         2.2           3.3          4.4  virginica
11          5.4         3.7           1.5          0.2     setosa
12          4.8         3.4           1.6          0.2     setosa
```

- 행의 인덱스 번호 10 위치에 새로운 행이 추가되었으며, 원래 데이터프레임에서 10 이후에 있던 행들은 하나씩 뒤로 밀렸다.
- pd.concat()에서 ignore_index=True를 지정한 이유는 새로 추가된 행으로 인해 일련번호의 연속성이 깨지므로, 이를 반영하여 레이블 인덱스를 새로 지정하기 위함이다.

02 데이터프레임 다루기

```
>>> # 여러 행의 추가
>>>
>>> ext = pd.DataFrame([[1.2, 3.5, 4.3, 3.1, 'setosa'],
...                      [2.1, 3.2, 2.3, 5.2, 'versicolor']],
...                     columns=df3.columns)
>>> ext
  Sepal_Length Sepal_Width Petal_Length Petal_Width     Species
0          1.2         3.5          4.3         3.1      setosa
1          2.1         3.2          2.3         5.2  versicolor
```

- 하나의 행이 아닌 여러 개의 행을 데이터프레임 뒷부분에 추가하는 경우, 특정 위치의 인덱스를 지정하는 방식은 사용할 수 없다.
- 이 경우에도 추가할 행들은 반드시 데이터프레임 형태로 작성되어야 한다.

02 데이터프레임 다루기

```
>> df3 = df3._append(ext, ignore_index = True)
>>> df3
    Sepal_Length Sepal_Width Petal_Length Petal_Width     Species
0            5.1         3.5          1.4         0.2      setosa
1            4.9         3.0          1.1         0.2      setosa
2            4.7         3.2          1.3         0.2      setosa
3            4.6         3.1          1.5         0.2      setosa
..           ...         ...          ...         ...         ...
149          5.9         3.0          5.1         1.8   virginica
150          1.1         4.5          3.4         2.2      setosa
151          1.2         3.5          4.3         3.1      setosa
152          2.1         3.2          2.3         5.2  versicolor

[153 rows x 5 columns]
```

- 데이터프레임 ext를 df3 뒤에 추가하기 위해서는 df3 = pd.concat([df3, ext], ignore_index=True)와 같이 해도 되고, 위의 예와 같이 _append() 메서드를 이용해도 된다.

02 데이터프레임 다루기

```
>>> # 뒤쪽에 컬럼의 추가
>>>
>>> new_col = df3.Petal_Length *10
>>> df3['new_col'] = new_col
>>> df3
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width     Species  new_col
0             5.1          3.5           1.4          0.2      setosa     14.0
1             4.9          3.0           1.1          0.2      setosa     11.0
2             4.7          3.2           1.3          0.2      setosa     13.0
3             4.6          3.1           1.5          0.2      setosa     15.0
..            ...          ...           ...          ...         ...      ...
149           5.9          3.0           5.1          1.8   virginica     51.0
150           1.1          4.5           3.4          2.2      setosa     34.0
151           1.2          3.5           4.3          3.1      setosa     43.0
152           2.1          3.2           2.3          5.2  versicolor     23.0

[153 rows x 6 columns]
```

- new_col은 판다스 시리즈 데이터이다.

02 데이터프레임 다루기

```
>>> # 중간에 컬럼의 추가
>>>
>>> df4 = df.copy()
>>> df4.insert(loc=2, column='new_col2', value=new_col)     # 2번째 위치에 추가
>>> df4
     Sepal_Length  Sepal_Width  new_col2  Petal_Length  Petal_Width    Species
0             5.1          3.5      14.0           1.4          0.2     setosa
1             4.9          3.0      11.0           1.4          0.2     setosa
2             4.7          3.2      13.0           1.3          0.2     setosa
3             4.6          3.1      15.0           1.5          0.2     setosa
..            ...          ...       ...           ...          ...        ...
146           6.3          2.5      50.0           5.0          1.9  virginica
147           6.5          3.0      52.0           5.2          2.0  virginica
148           6.2          3.4      54.0           5.4          2.3  virginica
149           5.9          3.0      51.0           5.1          1.8  virginica

[150 rows x 6 columns]
```

- 컬럼의 마지막이 아닌 중간에 새로운 컬럼을 추가하려면 insert() 메서드를 사용한다.

02 데이터프레임 다루기

```
>>> df4.insert(loc=2, column='new_col2', value=new_col)
```

표 4-2 insert() 메서드의 매개변수

매개변수	의미
loc	새로운 컬럼을 추가할 위치
column	추가할 컬럼의 이름
value	추가할 컬럼의 데이터(시리즈)

■ [여기서 잠깐!]

- insert() 메서드를 이용하여 마지막 열에 새로운 컬럼을 추가하는 작업도 가능하다.

02 데이터프레임 다루기

- 데이터프레임 객체에 대한 행과 컬럼의 삭제

```
[코드 4-9]
# 코드4-1에 이어서 실행
# 데이터프레임 객체에 대한 행과 컬럼의 삭제
df5 = df.copy()

# 행의 삭제
df5 = df5.drop(index=[1,3])
df5
df5 = df5.reset_index()
df5

# 컬럼의 삭제
df5 = df5.drop(columns='Petal_Length')
```

02 데이터프레임 다루기

```
>>> # 행의 삭제
>>>
>>> df5 = df5.drop(index=[1,3])
>>> df5
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
0             5.1          3.5           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
5             5.4          3.9           1.7          0.4     setosa
..            ...          ...           ...          ...        ...
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[148 rows x 5 columns]
```

- 연속된 행을 삭제하려면 df5.drop(index=range(5,11))과 같이 사용한다.

02 데이터프레임 다루기

- df5를 확인해 보면 인덱스 번호 1과 3이 비어 있는 것을 알 수 있다.
  이러한 경우, 표시되는 레이블 인덱스 번호와 절대 위치 인덱스 번호가 달라져 혼란을 줄 수 있으므로, 레이블 인덱스를 재설정할 필요가 있다.

```
>>> df5 = df5.reset_index()
>>> df5
     index  Sepal_Length  Sepal_Width  Petal_Length  Petal_Width    Species
0        0           5.1          3.5           1.4          0.2     setosa
1        2           4.7          3.2           1.3          0.2     setosa
2        4           5.0          3.6           1.4          0.2     setosa
3        5           5.4          3.9           1.7          0.4     setosa
..     ...           ...          ...           ...          ...        ...
144    146           6.3          2.5           5.0          1.9  virginica
145    147           6.5          3.0           5.2          2.0  virginica
146    148           6.2          3.4           5.4          2.3  virginica
147    149           5.9          3.0           5.1          1.8  virginica

[148 rows x 6 columns]
```

- reset_index() 메서드를 사용하면 인덱스가 초기화(reset)된다.
- 초기화 이전 인덱스는 첫 번째 컬럼으로 자동 생성된다.

02 데이터프레임 다루기

- reset_index() 메서드 사용 시, 초기화 이전의 레이블 인덱스 값이 컬럼으로 추가되는 것을 원치 않으면 drop=True를 매개변수로 입력한다.

```
df5 = df5.reset_index(drop=True)
```

02 데이터프레임 다루기

```
>>> # 컬럼의 삭제
>>>
>>> df5 = df5.drop(columns='Petal_Length')
>>> df5
     Sepal_Length  Sepal_Width  Petal_Width    Species
0             5.1          3.5          0.2     setosa
1             4.7          3.2          0.2     setosa
2             5.0          3.6          0.2     setosa
3             5.4          3.9          0.4     setosa
..            ...          ...          ...        ...
144           6.3          2.5          1.9  virginica
145           6.5          3.0          2.0  virginica
146           6.2          3.4          2.3  virginica
147           5.9          3.0          1.8  virginica
```

- 삭제할 컬럼이 여러 개이면 리스트로 묶어서 기술한다.

03 csv 파일 조회

▪ 개요

- 판다스 데이터프레임을 출력할 때, 행이나 컬럼이 많으면 중간 부분이 생략되어 표시된다. 이로 인해 개략적인 형태는 파악할 수 있지만, 전체 내용을 확인하기 어려울 때가 있다.
- 전체 내용을 확인하려면 엑셀 파일로 저장 후 열어볼 수도 있지만, VS Code 내에서 직접 확인할 수 있다면 더 편리하다.
- 이러한 경우, VS Code의 확장 기능 중 하나인 Excel Viewer를 이용하면 데이터프레임을 보다 쉽게 확인할 수 있다.

```
>>> df5
     Sepal_Length  Sepal_Width  Petal_Width    Species
0             5.1          3.5          0.2     setosa
1             4.7          3.2          0.2     setosa
2             5.0          3.6          0.2     setosa
3             5.4          3.9          0.4     setosa
..            ...          ...          ...        ...
144           6.3          2.5          1.9  virginica
145           6.5          3.0          2.0  virginica
146           6.2          3.4          2.3  virginica
147           5.9          3.0          1.8  virginica
```

03 csv 파일 조회

- Excel Viewer가 설치되지 않은 경우, VS Code의 확장 기능(Extensions) 탭에서 Excel Viewer를 검색한 후 설치한다.

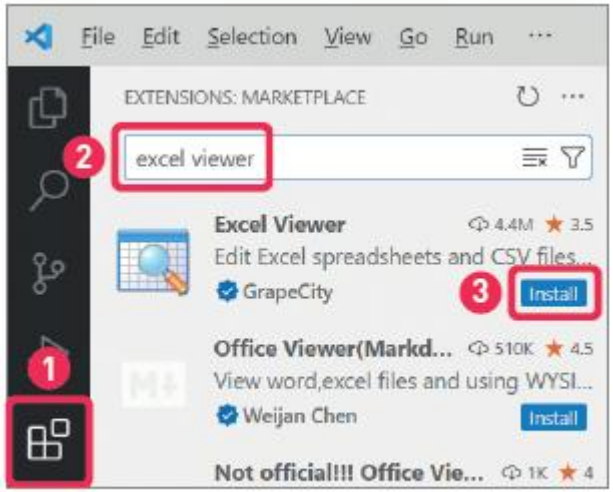
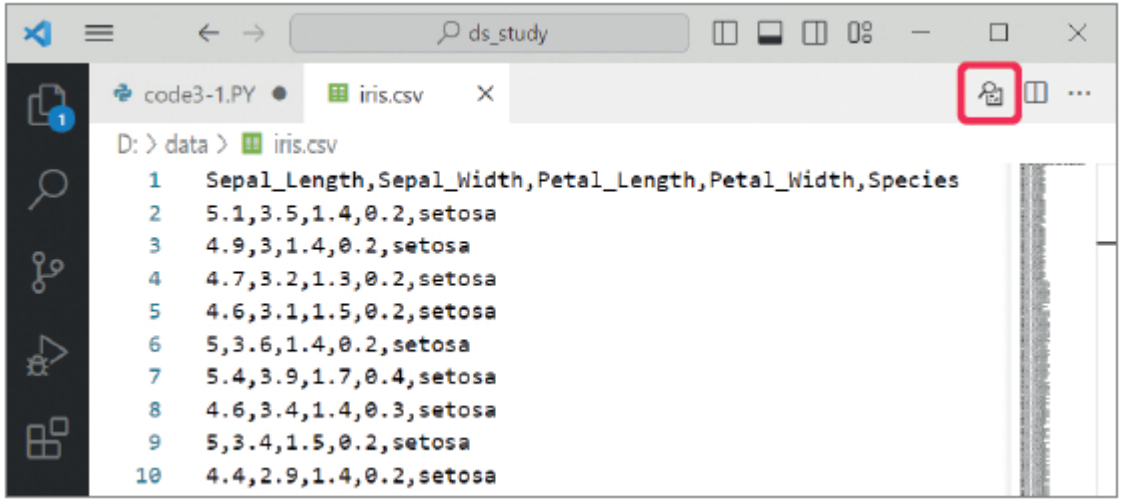


그림 4-4 Excel Viewer 설치

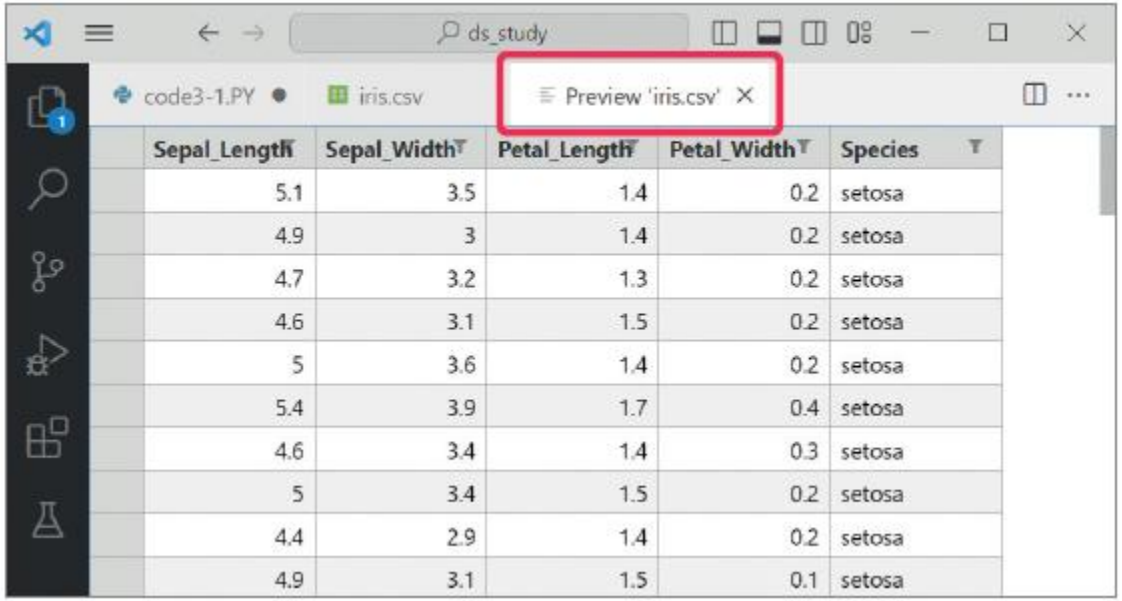
▪ Excel Viewer를 통해 iris.csv 파일 보기

- VS Code 메인 메뉴에서 [File] → [Open File] 메뉴를 선택하여 iris.csv 파일을 연다. 그러면 새로운 탭에서 iris.csv의 내용이 표시된다.



03 csv 파일 조회

- **Excel Viewer를 통해 iris.csv 파일 보기**
  - Preview 아이콘을 클릭하면 새로운 탭이 생성되며, iris.csv의 내용이 테이블 형태로 표시된다.
  - 이제 코드 편집 탭과 iris.csv 내용 표시 탭을 오가면서 작업할 수 있다.



Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa
4.6	3.4	1.4	0.3	setosa
5	3.4	1.5	0.2	setosa
4.4	2.9	1.4	0.2	setosa
4.9	3.1	1.5	0.1	setosa

♪♪ 실습해보기

- **제공한 GNI2014.csv 데이터셋에 대해 다음 문제를 해결하시오. 컬럼에 대한 설명은 다음과 같다.**

iso3	국가코드
country	국가명
continent	소속 대륙명
population	인구수
GNI	국민총소득

1. GNI2014.csv 파일을 읽어서 df에 저장하고 df 앞쪽 5개 행의 내용을 보이시오.
2. df의 행의 개수와 열의 개수를 각각 보이시오.
3. 소속 대륙의 종류를 보이시오.
4. ‘Norway’의 인구수와 국민총생산을 보이시오.
5. 유럽 대륙에 속한 국가들의 목록을 보이시오.

♪♪ 실습해보기

6. 인구수가 5천만 명 이상인 국가들의 목록을 보이시오
7. 유럽 국가들의 평균 인구수를 보이시오.
8. ‘Canada’보다 국민 총생산액이 많은 나라들의 평균 국민 총생산액을 보이시오
9. North America 국가들에 대해 인구가 10% 늘었을 때의 인구수를 보이시오.
10. 유럽 국가들에 대해 국가별 1인당 생산액(총생산액/인구수)을 보이시오
11. 유럽 국가들에 대해 국가별 1인당 생산액(총생산액/인구수)을 ‘PNP’라는 이름의 컬럼으로 해서 df에 추가한 후 앞쪽 5개의 행의 내용을 보이시오